Part 3 – Tejash: Our own CNN and ablations (Part B), and the website pages

What my part is

My part is Part B of the project: we built our own convolutional neural network (CNN) from scratch, trained it on the EuroSAT satellite tiles, and tested two changes to it. The baseline has three blocks of convolution, batch normalisation, ReLU and max pooling, with 32, 64 and 128 filters, followed by global average pooling and one linear layer that gives 10 class scores. We trained it with Adam (learning rate 0.001), cross-entropy loss, batches of 64 and at most 20 epochs, and we kept the epoch with the best validation accuracy. The two ablations were "BatchNorm off" and "two blocks instead of three", and each one was run with seeds 0, 1 and 2. The files I own are `src/models.py` (the function `build_scratch_cnn`), `notebooks/02_baseline_cnn.ipynb`, the figures and seed-0 history in `results/figures/`, and the Part B draft in `report/part_b_scratch_cnn_draft.md`. I also built the website pages: the landing page `frontend/Eyes_in_the_Sky_landing.html`, the classifier page `frontend/app.html`, and the API contract `frontend/API.md`. The main number to remember is that our scratch CNN reached a mean test accuracy of 92.7% over three seeds.

Definitions

Image tile (patch). A small square piece of a satellite image. Our tiles are 64 x 64 pixels with 3 colour channels (red, green, blue).

Channel. One layer of values in a stack of numbers. An RGB image has 3 channels. A feature map in a CNN also has channels, one per filter.

Convolution. A small grid of learned numbers slides over the image. At each position we multiply the grid with the pixels under it and add the results into one output number. This finds local patterns such as edges and textures.

Kernel and filter. The small learned grid is the kernel. Here every kernel is 3 x 3. People also say "filter" for one output channel of a convolution, so "32 filters" means 32 kernels that each make one feature map.

Stride. The number of pixels the kernel moves between positions. Our convolutions use the default stride of 1, so they look at every position.

Padding. Extra border pixels (zeros) added around the image before the convolution. With a 3 x 3 kernel, padding of 1 keeps the width and height the same.

Feature map. The grid of numbers produced by one filter. Each value says how strongly that filter's pattern appears at that spot.

ReLU. An activation function: it keeps positive numbers and turns negative numbers into 0. Without a non-linear step like this, stacked convolutions would act like one big linear operation.

Max pooling. For each 2 x 2 block of values, keep only the largest one. This halves the width and height and has no learned numbers.

Batch normalisation (BatchNorm). For each channel, it shifts and scales the values in a batch so their average is about 0 and their spread is about 1, then applies two learned numbers per channel (gamma and beta). It makes training steadier. In evaluation mode it uses running averages collected during training.

Global average pooling. Replace each whole feature map by the average of its values, so each map gives one number. Here it is written as `AdaptiveAvgPool2d(1)`, which also works if the input size changes.

Fully connected (linear) layer. Every input number is multiplied by its own learned weight, the results are added, and a bias is added. Our final layer maps 128 inputs to 10 outputs.

Logits. The raw output scores of the last layer, before they are turned into probabilities.

Softmax. A function that turns the 10 logits into 10 probabilities that add up to 1.

Cross-entropy loss. A loss that is small when the model gives the correct class a high probability and large when it gives it a low one. In PyTorch, `CrossEntropyLoss` takes the raw logits and applies softmax itself.

Backpropagation and gradients. After each batch, the loss is traced back through the network to work out how each weight should change to lower the loss. Those change directions are the gradients.

Optimiser, Adam. The rule that uses the gradients to change the weights. Adam keeps a running average of the gradients and of their squares, so each weight gets its own step size.

Learning rate. How big each weight change is. Ours is 0.001 for Adam.

Batch. A group of images processed together before one weight update. Our batch size is 64.

Epoch. One full pass through the training set. With 18,900 training images and batches of 64, one epoch has 296 updates (295 full batches and one batch of 20).

Train, validation and test sets. The training set (18,900 tiles) changes the weights. The validation set (4,050 tiles) is used to pick the epoch and to compare settings. The test set (4,050 tiles) gives the final score, and we scored it once per finished run. The split is 70/15/15 of 27,000 tiles.

Overfitting. The model learns the training images too closely. Training scores keep improving while validation scores stop improving or get worse.

Underfitting. The model is too simple or was not trained long enough, so both training and validation scores stay low.

Ablation. An experiment where you change one part of a model, keep everything else the same, and see how the result changes. It shows how much that part contributes.

Seed. A number that fixes the random choices in a run: initial weights, the order of the training images, and the augmentation choices. The same seed on the same machine gives the same run. Different seeds show how much results vary from run to run.

Mean and standard deviation. The mean is the average over seeds. We report the sample standard deviation (the spread across the three seeds, divided by n - 1). A small standard deviation means the seeds agreed.

Learning curve. A graph of loss or accuracy after each epoch, for both the training and the validation set. It shows whether the model is learning and whether it starts to overfit.

Parameters. The learned numbers in the model: convolution weights and biases, BatchNorm gamma and beta, and the final linear weights and biases.

Buffers. Numbers stored in the model that are not learned by gradients. BatchNorm keeps its running mean and running variance as buffers.

Accuracy. The fraction of images whose top predicted class is correct.

Macro-F1. The F1 score for each class, averaged with equal weight. F1 combines precision (how many predicted positives are right) and recall (how many true positives were found). Macro averaging stops big classes from hiding weak small ones.

Chance level. The accuracy a random guess would get. With 10 equally sized classes it is 10%.

Data augmentation. Making extra training images by changing existing ones in ways that keep the label. Our changes are 90-degree rotations and horizontal or vertical flips, which are safe for satellite tiles.

Normalisation (ImageNet mean and std). Subtracting a mean and dividing by a standard deviation for each colour channel, so the input numbers are in a standard range. We use the ImageNet numbers for all models so the comparison is fair.

Checkpoint and best-epoch selection. A checkpoint is a saved copy of the weights. We keep the weights from the epoch with the highest validation accuracy, not the last epoch.

model.train() and model.eval(). `train()` turns on training behaviour (BatchNorm uses batch statistics). `eval()` turns it off (BatchNorm uses running statistics). We call `eval()` before any validation or test scoring.

torch.no_grad(). A context that stops PyTorch from tracking gradients. Scoring does not need gradients, so it is faster and uses less memory.

DataLoader and num_workers. The DataLoader gives the model batches of images. `num_workers` is the number of extra worker processes that load images in parallel. We use 0, which means the main process loads them, because the one-worker setting gave a shared-memory error on this Windows laptop.

GPU and CUDA. A graphics card can run the matrix maths much faster than a CPU. CUDA is the NVIDIA system that lets PyTorch use it. Our laptop has an RTX 3050 with 6 GB.

Tensor. PyTorch's name for a multi-dimensional array of numbers. Image batches have the shape (N, C, H, W): number of images, channels, height, width.

Dropout. During training it sets a random share of values to 0 to reduce overfitting. We have a switch for it, but all our runs use 0.0, so it is off.

Grad-CAM. A heat map that shows which pixels pushed the model towards its chosen class. The website server makes it for the classifier page.

Confusion matrix. A 10 x 10 table. Rows are the true classes and columns are the predicted classes, so the diagonal is correct answers.

REST API, JSON, multipart form. A REST API is a set of web addresses (routes) that return data. JSON is the text format for that data. A multipart form sends a file and extra fields in one request, which is how the page uploads an image and the model name.

CORS. A browser rule that blocks a page on one address from calling a server on another address unless the server says it is allowed. Our server allows all origins, so the page works when opened from another address.

Code, line by line

Line numbers in the tables are counted inside each block. For `src/models.py` they are the real file line numbers. For the notebook, line 1 is the first line of the pasted cell.

1. `src/models.py`: `build_scratch_cnn` (lines 11 to 49)

The file also imports `torch.nn as nn` and `torchvision.models` at lines 7 and 8. The function is:

```
def build_scratch_cnn(
    num_classes=10,
    use_batchnorm=True,
    depth=3,
    dropout=0.0,
):
    """Build our CNN for 64x64 RGB images.
```

```
    The default settings are the baseline: 3 blocks with 32, 64 and 128 filters.
    Change one option at a time to make an ablation.
    """
    if depth not in (2, 3):
        raise ValueError("depth must be 2 or 3")
    if not 0 <= dropout < 1:
        raise ValueError("dropout must be between 0 and 1")
    if num_classes < 1:
        raise ValueError("num_classes must be positive")

    filter_counts = [32, 64, 128][:depth]
    layers = []
    input_channels = 3

    for output_channels in filter_counts:
        # Each block finds image features, then halves the image width and height.
        layers.append(nn.Conv2d(input_channels, output_channels, kernel_size=3, padding=1))
        if use_batchnorm:
            layers.append(nn.BatchNorm2d(output_channels))
        layers.append(nn.ReLU())
        layers.append(nn.MaxPool2d(kernel_size=2))
        input_channels = output_channels

    # Keep one value per feature map, whatever the final image size is.
    layers.append(nn.AdaptiveAvgPool2d(1))
    layers.append(nn.Flatten())
    if dropout > 0:
        layers.append(nn.Dropout(p=dropout))
    layers.append(nn.Linear(input_channels, num_classes))

    return nn.Sequential(*layers)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
11-16	Defines the function with four switches. The defaults are the baseline: 10 classes, BatchNorm on, depth 3, no dropout.	Default arguments, ablation switches	One function builds the baseline and every ablation, so the only difference between runs is the switch we change. Other options: a separate class per model (more code, more copies to keep in step) or a config file (too much for two switches).
22-23	Stops with an error unless depth is 2 or 3.	Input validation	A typo such as `depth=4` would otherwise build a model that is not in the plan. We check the value instead of letting it fail later. An `assert` would also work, but it can be switched off.
24-25	Dropout must be from 0 up to but not including 1.	Input validation	Dropout of 1 would switch off every value, so the model could not learn.
26-27	Class count must be at least 1.	Input validation	Stops a meaningless model.
29	Takes the first `depth` items of [32, 64, 128]. Depth 3 gives all three blocks; depth 2 gives 32 and 64.	Feature maps, slicing	Filters double as the network gets deeper, a common pattern. The depth-2 ablation just removes the last block. Alternative: list each block by hand, which is clearer but longer.
30-31	Creates an empty list for the layers and sets the input to 3 channels (red, green, blue).	Channels	`nn.Sequential` needs the layers in order, so we collect them in a list first.
33	Loops once per block (3 times for the baseline, 2 for depth 2).	Blocks	Each block has the same pattern, so one loop avoids writing it three times.
35	Adds a 3 x 3 convolution. `padding=1` keeps the width and height.	Convolution, kernel, padding	3 x 3 is the common small choice. A 5 x 5 kernel would have more weights for a similar result. A stride-2 convolution could replace max pooling, but we follow the plan.
36-37	Adds BatchNorm for that block, only if `use_batchnorm` is True.	Batch normalisation	This is the switch for the BatchNorm ablation. Other normalisations (GroupNorm, LayerNorm) exist, but the plan asked for BatchNorm.

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
38	Adds ReLU.	Activation	Lets the network learn non-linear patterns. Alternatives: LeakyReLU, GELU. ReLU is cheap and standard.
39	Adds 2 x 2 max pooling.	Max pooling	Halves the width and height (64 to 32 to 16 to 8), so later layers see a wider area. It has no weights. Average pooling is another option.
40	Sets the next block's input channels to this block's output channels.	Channels	Keeps the channel counts matching from one layer to the next.
42-43	Adds global average pooling. It turns each 8 x 8 feature map into one number.	Global average pooling	This gives 128 numbers and no weights. If we had flattened the 128 x 8 x 8 map instead, the linear layer would need 8,192 x 10 weights, many more than 1,280. The average also works for any input size.
44	Flattens (128, 1, 1) into a vector of 128 numbers.	Flatten	The linear layer needs a flat vector.
45-46	Adds dropout only when dropout is above 0.	Dropout	In our runs dropout is 0, so this layer is not added at all. It is kept as a switch for future tests.
47	Adds the final linear layer from 128 inputs to 10 outputs (the logits).	Fully connected layer, logits	One score per EuroSAT class. The softmax is not in the model; the loss function applies it.
49	Packs the layers into one `nn.Sequential` and returns it.	Sequential model	The model can be called like one function, and `.parameters()` finds all its weights.

2. Notebook `02_baseline_cnn.ipynb`, setup cell

```
from pathlib import Path
import json
import os
import sys

# Help CUDA reuse memory blocks on Windows. This does not change the model.
os.environ.setdefault('PYTORCH_CUDA_ALLOC_CONF', 'expandable_segments:True')

# Make imports work whether Jupyter starts here or in the notebooks folder.
project_root = Path.cwd()
if not (project_root / 'src').is_dir():
    project_root = project_root.parent
os.chdir(project_root)
sys.path.insert(0, str(project_root))
import time

import matplotlib.pyplot as plt
import pandas as pd
import torch
from sklearn.metrics import accuracy_score, f1_score

from src.dataset import get_loader
from src.models import build_scratch_cnn
from src.utils import FIGURES_DIR, METRICS_CSV, MODELS_DIR, get_device, set_seed

device = get_device()
print('Device:', device)
print('GPU:', torch.cuda.get_device_name(0) if device.type == 'cuda' else 'not available')
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1-4	Imports standard Python modules for paths, JSON, environment variables and the system path.	Python modules	Needed for file paths and for finding our `src` package.
6-7	Sets a CUDA memory setting if it is not already set.	GPU memory allocator	On a 6 GB laptop GPU, `expandable_segments` reduces memory fragmentation. It does not change the maths.

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
9-14	Finds the project root (the folder that contains `src`) and changes into it, then adds it to the import path.	Relative paths	The notebook works whether Jupyter starts in the root or in `notebooks/`. The data paths like `data/splits.csv` depend on this.
15-24	Imports time, plotting, pandas, torch, sklearn metrics and our own helpers (`get_loader`, `build_scratch_cnn`, paths, device, seed).	Shared code	We reuse the team's data loader and paths so every part uses the same split and folders.
26-28	Picks the GPU if one is available and prints its name.	Device, GPU	Training a CNN is much faster on the GPU. The printed output confirms it is the RTX 3050.

3. Notebook, settings cell

```
# These settings match the team plan.
BATCH_SIZE = 64
LEARNING_RATE = 0.001
MAX_EPOCHS = 20
SEEDS = [0, 1, 2]

# Change one setting for each ablation. Keep all other settings the same.
EXPERIMENTS = {
    'scratch_cnn': {'use_batchnorm': True, 'depth': 3, 'dropout': 0.0},
    'scratch_cnn_no_batchnorm': {'use_batchnorm': False, 'depth': 3, 'dropout': 0.0},
    'scratch_cnn_depth2': {'use_batchnorm': True, 'depth': 2, 'dropout': 0.0},
}

FIGURES_DIR.mkdir(parents=True, exist_ok=True)
MODELS_DIR.mkdir(parents=True, exist_ok=True)
assert Path('data/splits.csv').is_file(), 'Missing fixed split: data/splits.csv'
assert (Path('data/eurosat/2750')).is_dir(), 'Download EuroSAT using the README setup steps.'
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1-5	Writes the team's fixed settings: batch 64, learning rate 0.001, at most 20 epochs, seeds 0, 1, 2.	Hyper-parameters	Putting them at the top means one place to change them. These values come from the team plan so every model is compared on the same terms.
7-12	A dictionary of the three settings. Each entry is a set of arguments for `build_scratch_cnn`.	Ablation design	Each ablation differs from the baseline in exactly one switch. The dictionary keys become the model names in the results.
14-15	Creates the `results/figures` and `models` folders if they are missing.	Folders	The notebook saves files there, so the folders must exist.
16-17	Stops with a clear message if the split file or the EuroSAT images are missing.	Assertions	Fails early with a helpful message, instead of crashing later in the training loop.

4. Notebook, helper functions `score_model` and `train_one_model`

```
def score_model(model, loader, loss_function):
    """Return average loss and accuracy for one pass through a data loader."""
    model.eval()
    total_loss = 0.0
    all_labels = []
    all_predictions = []

    with torch.no_grad():
        for images, labels in loader:
            images = images.to(device)
            labels = labels.to(device)
            scores = model(images)
            total_loss += loss_function(scores, labels).item() * len(labels)
            all_labels.extend(labels.cpu().tolist())
            all_predictions.extend(scores.argmax(dim=1).cpu().tolist())

    average_loss = total_loss / len(loader.dataset)
```

```
    accuracy = accuracy_score(all_labels, all_predictions)
    return average_loss, accuracy


def train_one_model(model_name, options, seed, epochs=MAX_EPOCHS):
    """Train one setting and return the weights from its best validation epoch."""
    set_seed(seed)
    train_loader = get_loader(
        'train', seed=seed, augment=True, batch_size=BATCH_SIZE, num_workers=0
    )
    val_loader = get_loader('val', seed=seed, batch_size=BATCH_SIZE)

    model = build_scratch_cnn(**options).to(device)
    loss_function = torch.nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE, foreach=False)
    history = {'train_loss': [], 'train_accuracy': [], 'val_loss': [], 'val_accuracy': []}
    best_accuracy = -1.0
    best_epoch = 0
    best_weights = None

    for epoch in range(epochs):
        model.train()
        total_loss = 0.0
        correct = 0
        seen = 0

        for images, labels in train_loader:
            images = images.to(device)
            labels = labels.to(device)
            optimizer.zero_grad()
            scores = model(images)
            loss = loss_function(scores, labels)
            loss.backward()
            optimizer.step()

            total_loss += loss.item() * len(labels)
            correct += (scores.argmax(dim=1) == labels).sum().item()
            seen += len(labels)

        train_loss = total_loss / seen
        train_accuracy = correct / seen
        val_loss, val_accuracy = score_model(model, val_loader, loss_function)
        history['train_loss'].append(train_loss)
        history['train_accuracy'].append(train_accuracy)
        history['val_loss'].append(val_loss)
        history['val_accuracy'].append(val_accuracy)

        print(
            f'{model_name} | seed {seed} | epoch {epoch + 1:02d}/{epochs} | '
            f'train loss {train_loss:.4f}, train accuracy {train_accuracy:.3f} | '
            f'val loss {val_loss:.4f}, val accuracy {val_accuracy:.3f}'
        )

        if val_accuracy > best_accuracy:
            best_accuracy = val_accuracy
            best_epoch = epoch + 1
            best_weights = {
                name: value.detach().cpu().clone()
                for name, value in model.state_dict().items()
            }

    model.load_state_dict(best_weights)
    print(f'Best validation accuracy: {best_accuracy:.3f} at epoch {best_epoch}')
    return model, history, best_epoch
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
`score_model` 1-2	Defines a function that scores a whole loader and returns average loss and accuracy.	Evaluation function	Used for validation after each epoch, so the scoring code is written once.
`score_model` 3	`model.eval()` switches BatchNorm to its running averages.	Train vs eval mode	Scoring must not depend on the batch in front of it, so eval mode is required here.

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
`score_model` 8-15	Inside `no_grad()`, runs the model on each batch and collects the labels and predictions.	`torch.no_grad()`, argmax	No gradients are needed for scoring, which saves time and memory. `argmax` picks the highest score as the predicted class.
`score_model` 13	Adds loss times batch size to a running total.	Weighted average	The last batch can be smaller than 64, so multiplying by its size keeps the average correct.
`score_model` 17-19	Divides the total loss by the dataset size, computes accuracy with sklearn, and returns both.	Average loss, accuracy	Gives the same kind of numbers used in the learning curves.
`train_one_model` 24	Sets the seed.	Reproducibility	Makes the initial weights and the shuffle order repeatable for each seed.
`train_one_model` 25-28	Builds the training loader (with augmentation) and the validation loader (without).	Data loaders, augmentation	Augmentation is only for training, as the plan asks. Validation uses the fixed split with no changes.
`train_one_model` 30	Builds the model from the options and moves it to the GPU.	Model construction	The same function builds the baseline and each ablation.
`train_one_model` 31	Creates the cross-entropy loss.	Loss function	It works directly on the 10 logits and the true labels.
`train_one_model` 32	Creates Adam with learning rate 0.001.	Optimiser	The plan's settings. `foreach=False` picks a simpler per-weight loop of Adam. The notebook gives no reason; it changes speed and memory use, not the maths as far as I know. Check this before the viva.
`train_one_model` 33-36	Creates empty lists for the four curves and sets the best-epoch trackers.	Learning curves, checkpoint selection	The history is saved to JSON and plotted later.
`train_one_model` 38-39	Loops over the epochs and sets training mode.	Epoch, `model.train()`	Training mode lets BatchNorm use batch statistics.
`train_one_model` 44-51	For each training batch: clear old gradients, run the model, compute the loss, backpropagate, and take one Adam step.	Forward pass, backpropagation, optimiser step	This is the heart of learning. One epoch is 296 of these steps.
`train_one_model` 53-55	Adds the batch loss and the number of correct predictions to running totals.	Running training metrics	These are the training-set numbers for the curves. Note that they are measured during the epoch, while the weights are still changing and on augmented images.
`train_one_model` 57-63	Turns the totals into training loss and accuracy, scores the validation set, and stores all four numbers.	Learning curves	The validation pass uses the same epoch's weights, so the two curves line up.
`train_one_model` 65-69	Prints one line per epoch.	Logging	Lets us watch the run and copy the numbers.
`train_one_model` 71-77	If this epoch has the best validation accuracy so far, saves a copy of all weights.	Checkpoint selection	We use a strict `>`, so a tie keeps the earlier epoch. The test set is never used here. Alternative: save to disk each time.

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
`train_one_model` 79-81	Loads the best weights back into the model, prints the best epoch, and returns the model, history and epoch.	Model selection	The returned model is the best-validation model, not the last one.

5. Notebook, baseline run and learning curves (Lap 1)

```
checkpoint_path = MODELS_DIR / 'scratch_cnn_baseline_seed0.pth'
history_path = FIGURES_DIR / 'scratch_cnn_seed0_history.json'

# Reuse a finished local run if its checkpoint and history are both present.
if checkpoint_path.is_file() and history_path.is_file():
    with open(history_path, encoding='utf-8') as file:
        baseline_history = json.load(file)
    baseline_model = build_scratch_cnn(**EXPERIMENTS['scratch_cnn']).to(device)
    baseline_model.load_state_dict(
        torch.load(checkpoint_path, map_location=device, weights_only=True)
    )
    baseline_best_epoch = max(
        range(len(baseline_history['val_accuracy'])),
        key=baseline_history['val_accuracy'].__getitem__,
    ) + 1
    print('Loaded the completed local seed 0 baseline from its checkpoint and history.')
else:
    start_time = time.perf_counter()
    baseline_model, baseline_history, baseline_best_epoch = train_one_model(
        'scratch_cnn', EXPERIMENTS['scratch_cnn'], seed=0
    )
    elapsed_minutes = (time.perf_counter() - start_time) / 60
    print(f'Baseline run took {elapsed_minutes:.1f} minutes')
    torch.save(baseline_model.state_dict(), checkpoint_path)
    with open(history_path, 'w', encoding='utf-8') as file:
        json.dump(baseline_history, file, indent=2)

epochs = range(1, len(baseline_history['train_loss']) + 1)
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
axes[0].plot(epochs, baseline_history['train_loss'], label='Train')
axes[0].plot(epochs, baseline_history['val_loss'], label='Validation')
axes[0].set(title='Loss', xlabel='Epoch', ylabel='Cross-entropy')
axes[0].legend()
axes[1].plot(epochs, baseline_history['train_accuracy'], label='Train')
axes[1].plot(epochs, baseline_history['val_accuracy'], label='Validation')
axes[1].axhline(0.10, color='gray', linestyle=':', label='10% chance level')
axes[1].set(title='Accuracy', xlabel='Epoch', ylabel='Accuracy', ylim=(0, 1))
axes[1].legend()
fig.tight_layout()
fig.savefig(FIGURES_DIR / 'scratch_cnn_learning_curves.png', dpi=160)
plt.show()
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1-2	Sets the paths for the baseline weights and its history file.	Checkpoint files	Saving to known names lets later cells reuse the run.
4-15	If both files already exist, loads the history and the weights and finds the best epoch from the saved validation accuracies.	Reuse of finished runs	Saves about 20 epochs of training on reruns. `weights_only=True` loads only tensors, which is safer. `max` with `range` returns the first index with the highest value, so it matches the `>` rule in `train_one_model`.
16	Prints that the run was reused.	Logging	Makes it clear the output is from the saved run, not a new one.
17-26	Otherwise trains seed 0 from scratch, times it, saves the weights and the history.	Training and saving	Saves a checkpoint and JSON so the figures can be remade without retraining.

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
28-38	Draws two panels: loss (train and validation) and accuracy (train and validation), with a dotted 10% line for chance.	Learning curves	Two panels make it easy to see both loss and accuracy. The chance line shows the model is far above guessing.
39-41	Saves the figure at 160 dpi and displays it.	Figure export	The figure goes into `results/figures/` for the report and the website.

The notebook output for this cell says the run was loaded from the saved files, not retrained. So the printed numbers you see come from the earlier training run, and the curve figure was redrawn from the same history.

6. Notebook, ablation sweep (Lap 2)

```
# Run the full Part B comparison. Completed runs are read from metrics.csv on reruns.
RUN_ABLATION_SWEEP = True

if RUN_ABLATION_SWEEP:
    all_results = (
        pd.read_csv(METRICS_CSV)
        if METRICS_CSV.exists()
        else pd.DataFrame(columns=['model', 'fraction', 'seed', 'accuracy', 'macro_f1'])
    )
    rows = []
    for model_name, options in EXPERIMENTS.items():
        for seed in SEEDS:
            already_done = (
                (all_results['model'] == model_name)
                & (all_results['fraction'] == 1.0)
                & (all_results['seed'] == seed)
            ).any()
            if already_done:
                print(f'Skip {model_name}, seed {seed}: test result is already saved.')
                continue

            if model_name == 'scratch_cnn' and seed == 0:
                # Seed 0 was trained above; reuse it and score its test split once.
                model = baseline_model
                best_epoch = baseline_best_epoch
            else:
                model, history, best_epoch = train_one_model(model_name, options, seed)
            test_loader = get_loader('test', seed=seed, batch_size=BATCH_SIZE)
            test_labels = []
            test_predictions = []
            model.eval()
            with torch.no_grad():
                for batch_number, (images, labels) in enumerate(test_loader, start=1):
                    scores = model(images.to(device))
                    test_labels.extend(labels.tolist())
                    test_predictions.extend(scores.argmax(dim=1).cpu().tolist())
                    if batch_number % 10 == 0:
                        print(f'test batches scored: {batch_number}')

            row = {
                'model': model_name,
                'fraction': 1.0,
                'seed': seed,
                'accuracy': accuracy_score(test_labels, test_predictions),
                'macro_f1': f1_score(test_labels, test_predictions, average='macro'),
            }
            rows.append(row)
            torch.save(
                model.state_dict(),
                MODELS_DIR / f'{model_name}_seed{seed}.pth',
            )
            same_run = (
                (all_results['model'] == model_name)
                & (all_results['fraction'] == 1.0)
                & (all_results['seed'] == seed)
            )
            all_results = all_results.loc[~same_run]
            all_results = pd.concat([all_results, pd.DataFrame([row])], ignore_index=True)
            all_results.to_csv(METRICS_CSV, index=False)
```

```
            print(f"Test result (best epoch {best_epoch}): {row}")

    comparison = all_results[
        all_results['model'].isin(EXPERIMENTS)
        & (all_results['fraction'] == 1.0)
        & all_results['seed'].isin(SEEDS)
    ]
    summary = comparison.groupby('model')[['accuracy', 'macro_f1']].agg(['mean', 'std'])
    display(summary)

    fig, axes = plt.subplots(1, 2, figsize=(10, 4))
    for axis, metric, title in zip(
        axes, ['accuracy', 'macro_f1'], ['Test accuracy', 'Test macro-F1']
    ):
        grouped = comparison.groupby('model')[metric]
        means = grouped.mean()
        spreads = grouped.std().fillna(0)
        axis.bar(means.index, means.values, yerr=spreads.values, capsize=4)
        axis.set(title=title, ylabel=metric.replace('_', ' '), ylim=(0, 1))
        axis.tick_params(axis='x', rotation=25)
    fig.tight_layout()
    fig.savefig(FIGURES_DIR / 'scratch_cnn_ablation_comparison.png', dpi=160)
    plt.show()
else:
    print('Ablation sweep not run. Set RUN_ABLATION_SWEEP = True when ready.')
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1-2	Comment and a flag that turns the whole sweep on or off.	Flag	Lets the notebook be run without the long sweep.
4-9	Loads the saved `metrics.csv` if it exists, or makes an empty table with the right columns.	Persistent results	Results survive restarts. The table is shared with the data-efficiency study, which uses other fraction values.
10-12	Loops over the 3 settings, and for each one over seeds 0, 1, 2.	Grid of experiments	Nine runs in total: 3 settings times 3 seeds.
13-20	Checks whether that setting and seed already have a test result in the table; if so, prints a skip message and moves on.	Resumable runs	If the notebook stops halfway, rerunning does not repeat finished work. The filter `fraction == 1.0` keeps only the full-data rows.
22-25	For the baseline with seed 0, reuses the model trained in Lap 1 instead of training it again.	Reuse	Avoids training the same run twice.
26-27	Otherwise trains a new model with `train_one_model`.	Training	Each run selects its epoch by validation accuracy only.
28-38	Scores the test set once with the chosen weights. Stores the labels and predictions.	Test set, `no_grad`	The test set is used only here, after the run is finished. It is not used to choose the epoch.
40-46	Makes one row with the setting name, seed, test accuracy and macro-F1.	Accuracy and macro-F1	Macro-F1 gives each class equal weight, which is fair when some classes are smaller.
47-51	Saves the weights for this run as `<model>_seed<n>.pth`.	Saving models	Lets later notebooks reuse the models.
52-59	Replaces any old row for this setting and seed, then saves the table to CSV.	Upsert	A rerun cannot make duplicate rows. Saving after every run protects finished work.

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
60	Prints the best epoch and the row.	Logging	Gives a record of each run.
62-66	Keeps only the three settings, full data, and seeds 0, 1, 2.	Filtering	The table may hold other rows from the data-efficiency study.
67-68	Groups by setting and computes the mean and standard deviation of accuracy and macro-F1.	Mean and standard deviation	This creates the ablation table in the report. Pandas uses the sample standard deviation.
70-79	Draws two bar charts (accuracy and macro-F1) with error bars for the spread across seeds.	Error bars	Error bars show whether differences are bigger than the seed-to-seed spread, which is the question the plan asks.
80-82	Saves the comparison figure and displays it.	Figure export	Output goes to `results/figures/scratch_cnn_ablation_comparison.png`.
83-84	Prints a message if the sweep is turned off.	Flag	Keeps the notebook readable when the flag is False.

The saved output for this cell shows all nine runs as "Skip", so the table and plot were read from `results/metrics.csv` in that session, not retrained. Two things to know about this cell. First, the plan asks for this loop to call the shared `train()` function from `src/train.py`. The notebook still uses its own helper, because the notebook text says `train.py` is a TODO. My search of `src/train.py` for "TODO" found nothing, so check whether the shared trainer is now finished. Second, the plan's optional third ablation (augmentation off) was not run.

The architecture layer by layer

Shapes are per image (channels x height x width). A batch adds a first number N, so a batch of 64 images has the shape 64 x 3 x 64 x 64. The rows below come from running the real code on a 3 x 64 x 64 input. Dropout is not in the list because `dropout=0.0`.

#	Layer	Input shape	Output shape	Parameters
0	Conv2d, 3 to 32, kernel 3, padding 1	3 x 64 x 64	32 x 64 x 64	896
1	BatchNorm2d (32)	32 x 64 x 64	32 x 64 x 64	64
2	ReLU	32 x 64 x 64	32 x 64 x 64	0
3	MaxPool2d, 2 x 2	32 x 64 x 64	32 x 32 x 32	0
4	Conv2d, 32 to 64, kernel 3, padding 1	32 x 32 x 32	64 x 32 x 32	18,496
5	BatchNorm2d (64)	64 x 32 x 32	64 x 32 x 32	128
6	ReLU	64 x 32 x 32	64 x 32 x 32	0
7	MaxPool2d, 2 x 2	64 x 32 x 32	64 x 16 x 16	0
8	Conv2d, 64 to 128, kernel 3, padding 1	64 x 16 x 16	128 x 16 x 16	73,856
9	BatchNorm2d (128)	128 x 16 x 16	128 x 16 x 16	256
10	ReLU	128 x 16 x 16	128 x 16 x 16	0
11	MaxPool2d, 2 x 2	128 x 16 x 16	128 x 8 x 8	0
12	AdaptiveAvgPool2d(1), global average pooling	128 x 8 x 8	128 x 1 x 1	0
13	Flatten	128 x 1 x 1	128	0
14	Linear, 128 to 10	128	10 (logits)	1,290
	Total trainable parameters			**94,986**

How 94,986 is computed. Three rules cover every learned number:

- A convolution has out x in x kernel_height x kernel_width weights, plus one bias per output channel. Block 1: 32 x 3 x 3 x 3 + 32 = 864 + 32 = 896. Block 2: 64 x 32 x 3 x 3 + 64 = 18,432 + 64 = 18,496. Block 3: 128 x 64 x 3 x 3 + 128 = 73,728 + 128 = 73,856.
- A BatchNorm layer has two learned numbers per channel (gamma and beta). So 2 x 32 = 64, 2 x 64 = 128, and 2 x 128 = 256.
- A linear layer has in x out weights plus out biases: 128 x 10 + 10 = 1,290.

Adding them: 896 + 64 + 18,496 + 128 + 73,856 + 256 + 1,290 = 94,986. PyTorch's own count, `sum(p.numel() for p in model.parameters())`, gives the same number.

BatchNorm also stores a running mean and a running variance per channel. These are buffers, not parameters. They add 2 x 224 = 448 stored numbers, plus 3 counters, which is 451 buffer values in total. Adam never updates them; BatchNorm updates them itself during training.

A rough compute figure, from the convolution sizes (my own estimate): about 41.3 million multiply-adds per image, of which about 18.9 million are in each of blocks 2 and 3.

The two ablation models, same rules:

- No BatchNorm: 94,986 minus the 448 gamma and beta values = **94,538**.
- Depth 2: block 1 (896 + 64) + block 2 (18,496 + 128) + linear 64 to 10 (640 + 10 = 650) = **20,234**.

Note that the depth-2 model is about a fifth of the size of the baseline, so the depth change also reduces capacity. This matters for the ablation discussion below.

The ablations

What changed. Each ablation changes one switch and keeps everything else the same:

Run	Switch changed	What the network looks like
Baseline	none	3 blocks, BatchNorm on, dropout 0, 94,986 parameters
No BatchNorm	`use_batchnorm=False`	The same 3 blocks, but the three BatchNorm layers are removed, 94,538 parameters
Depth 2	`depth=2`	Only blocks 1 and 2 (32 and 64 filters). The global average pool now sees 64 values, 20,234 parameters

Why one thing at a time. If we changed two things together, we could not tell which one caused a difference. Because the switches live in one function, each run differs from the baseline only in the switch.

Results. Test scores, mean and sample standard deviation over seeds 0, 1 and 2 (from `results/metrics.csv`):

Setting	Test accuracy (mean ± std)	Test macro-F1 (mean ± std)	Accuracy per seed (0, 1, 2)
Baseline	92.72% ± 0.45%	92.48% ± 0.51%	93.14, 92.77, 92.25
No BatchNorm	88.95% ± 0.63%	88.45% ± 0.64%	88.22, 89.31, 89.33
Depth 2	85.31% ± 1.43%	84.18% ± 1.78%	84.44, 84.52, 86.96

Changes from the baseline:

- No BatchNorm: accuracy down 3.76 points, macro-F1 down 4.03 points.
- Depth 2: accuracy down 7.41 points, macro-F1 down 8.30 points.

Is the difference bigger than the seed spread? For both ablations the answer is yes, with 3 seeds.

- The ranges do not overlap. The worst baseline seed (92.25%) is above the best no-BatchNorm seed (89.33%) and the best depth-2 seed (86.96%).
- The baseline beats each ablation on every seed, by at least 2.92 points for no-BatchNorm (pairs: 4.92, 3.46, 2.92) and at least 5.29 points for depth 2 (pairs: 8.70, 8.25, 5.29).
- The gap between the means, divided by the combined spread (the square root of the sum of the squared standard deviations), is about 4.8 for BatchNorm and about 4.9 for depth 2. That is a rough check, not a formal significance test. With three seeds a formal test would have very few degrees of freedom, so we report it as a clear but informal result.

The seeds are shared across settings (seed 0 is seed 0 for each model), but the architectures differ, so the initial weights are not the same. Do not claim more than "the same seed number".

What this does and does not prove.

- BatchNorm helps this model on this data, with this learning rate and batch size. It does not show that BatchNorm helps every CNN.
- The depth ablation changes two things together: the number of blocks and the number of parameters (95k to 20k). So we cannot say that depth alone is responsible. It is "a smaller, shallower model", which is weaker.
- We picked the baseline and the ablations before running them, and we did not tune anything on the test set.

How to read the learning curves

The figure `results/figures/scratch_cnn_learning_curves.png` has two panels, both for seed 0 over 20 epochs. The numbers below come from `results/figures/scratch_cnn_seed0_history.json`.

Epoch	Train loss	Val loss	Train accuracy	Val accuracy
1	0.912	0.667	69.5%	77.9%
5	0.419	0.574	85.7%	80.7%
10	0.283	0.335	90.2%	87.9%
14	0.240	0.248	91.6%	91.6%
16	0.230	0.560	91.9%	83.5%
20	0.204	0.224	93.0%	92.6%

What the panels show:

- **Loss.** Both losses fall from about 0.9 and 0.7 to about 0.2. Validation loss is noisy: it jumps to 0.80 at epoch 2, to 0.56 at epoch 16 and to 0.40 at epoch 19. These spikes mean the model sometimes becomes less confident on validation images in one epoch, then recovers. The learning rate is constant (no schedule), which is one likely cause.
- **Accuracy.** Training accuracy rises steadily. Validation accuracy also rises, but it dips at epochs 2, 15 and 16 (83.5% at epoch 16). It finishes highest at epoch 20 with 92.64%, so that epoch is the saved checkpoint.
- **The gap.** At epoch 10 the gap is about 2.3 points (90.2% train, 87.9% val). At epoch 14 the two accuracies are equal. At epoch 20 the gap is about 0.3 points. So there is no sign of overfitting at the end, because validation kept up with training. Overfitting would show as a widening gap, with validation flat or falling while training keeps rising.
- **Chance.** The dotted line at 10% is chance for 10 balanced classes. Even in epoch 1, training accuracy (69.5%) and validation accuracy (77.9%) are far above guessing.

Two cautions for the viva. Training accuracy is measured during each epoch, on augmented images, while the weights are still changing, so it is not the same as scoring the training set at the end. And the curves are for one seed; the seed-to-seed spread is in the ablation table, not in the curves.

Website pages Tejash built

The classifier page: `frontend/app.html`

The classifier page lets a user drop, click to choose, or paste an image (or click one of four sample patches). It shows:

- the predicted class and a confidence ring (the top probability),
- a bar chart of the top 3 classes with their probabilities,
- a Grad-CAM heat map over the image, with a slider for its strength,
- a short "What it looks for" sentence, taken from a fixed list of 10 hand-written sentences, one per class.

The page also has a Results tab (`#results`). It shows the test accuracy and macro-F1 with a counting animation, a 10 x 10 confusion matrix (rows are true classes, columns are predictions, blue is correct), and an accuracy-against-training-data chart with a line for the scratch CNN and one for the pre-trained model.

Two modes. The page starts in **demo mode** (the status pill says "Demo mode: simulated results") when the address has no `?api=`. In demo mode the predictions come from a fake function that uses the average colour of the image, and the heat map is a generated blob. This is not our CNN. When the page is opened as `app.html?api=http://localhost:8000` and `/api/models` answers, the pill says "Backend connected" and every prediction comes from the server. If a prediction fails in live mode, the page shows an error and then displays a simulated result, so the team should watch the status pill during the demo.

How it calls the API. The routes are in `frontend/API.md`, and the server that implements them is `Backend/app/server.py`:

Call from the page	Route	What the page sends	What it uses from the reply
On load (live mode only)	`GET /api/models`	nothing	`models` list, filled into the model dropdown as `id` and `label`
When an image is chosen	`POST /api/predict`	multipart form: `image` (the file) and `model` (the id from the dropdown)	`label`, `confidence`, `top3`, `gradcam` (a PNG as a data URL)
Opening the Results tab (live mode)	`GET /api/metrics`	nothing	`accuracy`, `macro_f1`, `confusion` (10 x 10), `efficiency` (`fractions`, `scratch`, `transfer`)

Some details to know:

- `/api/predict` returns the top 3 classes with a softmax probability each, and a Grad-CAM overlay the same size as the input. The page sets it as the `src` of an image and overlays it on the uploaded picture.
- The class names must match the EuroSAT folder names exactly, because the page looks up its "What it looks for" sentence by name.
- The server allows all origins (CORS), which the page needs when it is served from a different address.
- The server finds models from files in `Backend/models/`. It expects names like `<model>_frac1_seed0.pth`. The API.md example lists only `resnet18`, while the page's demo dropdown uses an id called `scratch`. The real id must come from the server. I read the three route definitions and the predict code, but not the helper that lists the weights, so I cannot confirm the exact ids.

The landing page: `frontend/Eyes_in_the_Sky_landing.html`

The landing page is one static HTML file of about 810 lines and 222 KB. It has no calls to the API. Its numbers are written into the page. The structure, from top to bottom:

- a hero with the title "Eyes in the Sky" and two buttons, "Start the story" (to the story chapters) and "Try the classifier" (to `app.html`);
- a navigation bar, plus a chapter list (an "orbit" nav) that highlights the current chapter as you scroll;

- "Ten kinds of ground to tell apart" (`#classes`), with a tile for each class;
- "How much labelled data does it need?" (`#data`), a data-efficiency explorer that draws the scratch CNN and the pre-trained model from a `DATA` object in the script;
- "Six phases, one pipeline" (`#pipeline`), with the six phases: understand the data, build a baseline CNN, borrow pre-trained eyes, measure data efficiency, evaluate and explain, and document and submit;
- a team section listing each member and what they own;
- "Frequently asked questions" (`#desk`).

The page animates its content with scroll and timed effects. Many of its visual details are not my focus here; the important point is that the page does not check the API.

Things to check before the viva

These are gaps and mismatches I found while reading. Fix them before the Friday demo.

1. **Draft and log numbers do not match the saved history.** The draft and the log say the best validation accuracy was 93.01% at epoch 20, the final training accuracy 92.88%, the validation loss at the end 0.210, and the training loss at the end 0.203. The saved JSON (`scratch_cnn_seed0_history.json`) gives 92.64%, 92.97%, 0.224 and 0.204. I could not find which run the draft numbers came from. Use the JSON values, or re-run, and correct the draft.
2. **The landing page team list swaps two people.** In the `TEAM` list, Tejash is marked "Data and EDA" and Shrivalli is marked "Scratch CNN and ablations". These should be the other way round. The spelling "Shrivalli" also differs from "Srivalli" in other files.
3. **The landing page scratch number is seed 0, not the mean.** The explorer shows "Scratch CNN 93.1", which is the seed-0 test accuracy (93.14%). Our three-seed mean is 92.7%. Say so, or use the mean.
4. **Weight file names may not match the server.** The notebook saves `scratch_cnn_baseline_seed0.pth` and `scratch_cnn_seed0.pth`. The server looks for `<model>_frac1_seed0.pth`. The `models/` folder has `scratch_cnn_frac1_seed0.pth`, which this notebook does not write. Check where that file came from before the demo.
5. **The shared trainer note may be out of date.** The notebook says `src/train.py` is a TODO. My search found no "TODO" in that file. If the shared `train()` is finished, the plan asks us to connect the Lap 2 sweep to it.
6. **The augmentation-off ablation was not run.** It was optional in the plan. Say so if asked.
7. **The sample patches are drawn colour blocks,** not real EuroSAT tiles. They are useful for showing the model can give confident answers on images outside its world, but do not present them as data.
8. **The metrics CSV is shared.** Rows with fractions below 1.0 belong to the data-efficiency study (Nitin's part). My comparison filters to fraction 1.0 and seeds 0, 1 and 2.